

- [CPSC 375: High-Performance Computing](#)

[CPSC 375: High-Performance Computing](#)

Spring 2026

- [Syllabus](#)
- [Schedule](#)
- [Upload](#)
- [The Cluster Project](#)

Homework 30

NOTE: You are not required to hand in the following exercises, but you are strongly encouraged to complete them to strengthen your understanding of the concepts covered in class.

1. Consider the iterative, row-oriented algorithm (the *ijk* version). Explain why performance (in megaflops) drops sharply once the matrix size exceeds 100-200, as shown in the graph on slide 4. Also, define stride in the context of accessing B in this algorithm. How does a large stride specifically impact the CPU cache and the number of memory transfers required?.
2. Loop reordering introduces the *ikj* version of matrix multiplication. Provide the specific line of code from the *ikj* version that demonstrates spatial locality for both matrix C and matrix B. Explain why the variable *r* is used to store *a[i][k]* outside of the innermost loop. How many times is this specific element of A reused before the next value of *k* is loaded?.
3. Tiling (blocking) is used to manage cache capacity. Given a cache size *M*, what is the communication complexity (number of memory transfers) for blocked matrix multiplication compared to the standard approach? In addition, tiling uses a MIN function. Explain the practical purpose of this function when the matrix size *n* is not perfectly divisible by the `block_size`.
4. Strassen's algorithm improves matrix multiplication efficiency by reducing the number of multiplications required for a 2-by-2 block from eight to seven.
 - A. Use the specific formulas provided on slides 16 and 17 to calculate the value of the quadrant C_{12} given only the intermediate products P_1 through P_7 , noting that the quadrant is defined as $C_{12} = P_1 + P_2$.
 - B. Implement Strassen's algorithm using a divide-and-conquer approach that recursively splits *n*-by-*n* matrices into four sub-matrices of size *n*/2-by-*n*/2 until the base case is reached. By recursively applying these seven multiplication formulas, the algorithm achieves a reduced work complexity of approximately $O(n^{2.81})$ compared to the traditional $O(n^3)$ method.
5. Compare the two optimization techniques discussed on slide 12. If a matrix is significantly larger than the cache, why might loop reordering alone fail to reach peak performance? Which type of locality (spatial or temporal) is specifically improved by blocking that is not explicitly managed by simple reordering? Refer to the diagrams on slide 13 to support your answer.

- **Welcome: Sean**

- [LogOut](#)

